

CIRCUITWEAVE: Topology–Behavior Alignment for Executable Multimodal RTL Generation

Jiahao Feng

Hangzhou International Innovation Institute, Beihang
University
Hangzhou, China
fengjiahao@buaa.edu.cn

Zhiwei Xie

Hangzhou International Innovation Institute, Beihang
University
Hangzhou, China
xiezhwei@buaa.edu.cn

Haiyan Qin

National College for Excellent Engineers, Beihang
University
Beijing, China
Jingjinji National Center of Technology Innovation
Beijing, China
haiyanq@buaa.edu.cn

Wang Kang*

Hangzhou International Innovation Institute, Beihang
University
Hangzhou, China
School of Integrated Circuit Science and Engineering,
Beihang University
Beijing, China
wkang@buaa.edu.cn

Abstract

Text-only LLMs generate RTL from natural-language specifications, but prose can leave connectivity, register boundaries, and state–output relations implicit even when interfaces and cycle-level behavior are specified. Schematics can make these structural relations explicit and thereby complement the behavioral constraints conveyed by text. Yet simply adding an image creates a fusion challenge: direct multimodal decoding does not explicitly separate the evidence roles of text and schematics or make missing and conflicting constraints explicit before code generation. We present CIRCUITWEAVE, a contract-mediated multimodal framework that extracts a topology contract from the schematic and a behavior contract from the text. It fuses these records into a circuit contract that serializes correspondences, missing evidence, and conflicts, then generates RTL only from this contract. A joint objective supervises both contracts, serialized fusion, contract-conditioned RTL generation, and reverse reconstruction of covered contract fields from reference RTL. We construct 5,000 executable-qualified packages, each containing a specification, generated schematic, structured contracts, reference RTL, and self-checking testbench, and use the training split to adapt Qwen3.5-4B with LoRA. On VerilogEval-Human, CIRCUITWEAVE reaches 46.60% pass@1, 61.49% pass@5, and 65.39% pass@10. These point estimates are 8.46, 5.85, and 2.57 points above those of the same adapted checkpoint without the schematic. On RTLLM, it reaches 40.00%, 48.00%, and 52.00%, two points above the adapted text-only condition at each cutoff. The dataset is publicly available at <https://huggingface.co/datasets/fengjiahao0421/CircuitWeave>.

CCS Concepts

• **Hardware** → **Hardware description languages and compilation; Electronic design automation**; • **Computing methodologies** → *Machine learning*.

*Corresponding author: Wang Kang (wkang@buaa.edu.cn).

Keywords

RTL generation, electronic design automation, Verilog, multimodal learning, circuit contracts, cross-modal alignment, executable verification

1 Introduction

Prior LLM-based RTL methods include English-to-Verilog translation, domain adaptation, and hierarchical instruction construction [14, 18, 25], as well as structured training tasks and reasoning with executable feedback [9, 26]. Executable evaluation is stricter than syntax generation: a candidate must satisfy the interfaces, widths, clock/reset semantics, and cycle-level behaviors exercised by a functional testbench. VerilogEval and RTLLM evaluate candidates through compilation and simulation [8, 11]. Intended circuit organization remains a separate structural target because black-box simulation need not verify it.

Text and schematics provide complementary views of hardware intent. Text can specify ports, bit widths, reset polarity, clocks, latency, corner cases, and expected behavior, but may leave connectivity and state organization implicit. Schematics can expose hierarchy, datapaths, register boundaries, and state transitions, but may omit exact interface and timing details. Chang et al. report benchmark cases in which diagrams provide hardware information absent from language [1]. Thus, for independently authored specifications, neither modality fully substitutes for the other. In our controlled benchmark study, however, each schematic is generated from the same description and provides a structured visual restatement rather than an independent specification.

These roles motivate a more precise question than whether an image helps: under what training conditions does a description-derived schematic improve executable RTL generation? A single text+image score cannot distinguish RTL-specific adaptation from image conditioning. Without an explicit division of evidence, direct fusion need not preserve which constraints come from text and

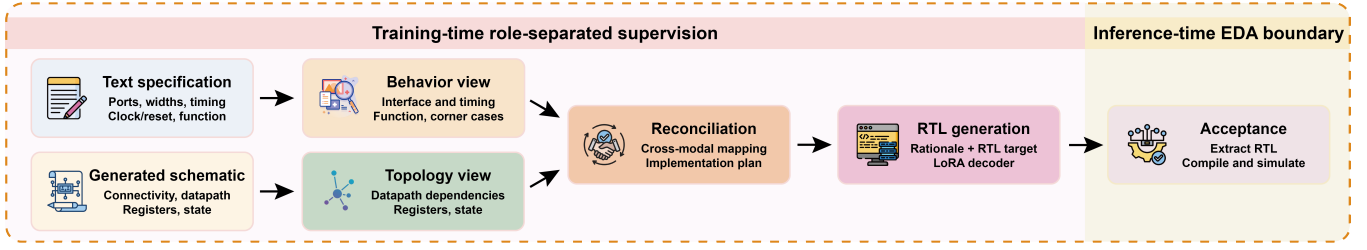


Figure 1: Conceptual overview of CIRCUITWEAVE. Text and the generated schematic provide behavior and topology evidence, which reconciliation combines into fused contract z . The RTL-generation call receives only z , and only extracted RTL reaches the hidden EDA boundary. The source-restricted calls follow Eq. 1; the training-only reverse-reconstruction task is omitted.

which come from the schematic. A model may follow visual connectivity while violating a textual width, or match the interface while inserting an incorrect register boundary. We therefore encode the two evidence roles separately and vary checkpoint state and image input in a matched design.

We formulate this setting as topology-behavior contract mediation and instantiate it in CIRCUITWEAVE. The topology contract z_o records components, connectivity, datapath operators, and sequential structure from the schematic. The behavior contract z_t records interfaces, widths, clock/reset semantics, latency, corner cases, and required function from the text. The fusion stage serializes correspondences, missing evidence, and conflicts in a circuit contract z . A separate stage generates RTL conditioned on z ; the raw modalities are excluded from the RTL-generation call. Figure 1 gives the conceptual overview. Equation 1 and Listing 1 specify the source-restricted calls and fused-contract-only RTL stage. We jointly optimize contract extraction, serialized fusion, contract-conditioned RTL generation, and reverse reconstruction of covered contract fields from reference RTL, and adapt Qwen3.5-4B using LoRA [6, 16].

We evaluate this setting using 5,000 executable-qualified packages and a matched 2×2 comparison of checkpoint state and input condition. On VerilogEval-Human, the adapted text+image condition reaches 46.60% pass@1, 61.49% pass@5, and 65.39% pass@10. At pass@1, its image-input difference is +8.46 points, compared with -3.01 points for the base checkpoint. On RTLLM, the adapted text+image condition reaches 40.00%, 48.00%, and 52.00%, two points above the adapted text-only condition at each cutoff. Across these descriptive point estimates, the image-input difference varies by checkpoint state, benchmark, and sampling cutoff (Tables 4 and 5).

This work makes three main contributions:

- We introduce contract-mediated multimodal RTL generation, which constructs modality-specific topology and behavior contracts, fuses them into a circuit contract, and decodes RTL from that fused record. The joint objective supervises contract extraction, serialized fusion, RTL generation, and reverse reconstruction of covered contract fields.
- We construct 5,000 executable-qualified design-verification packages through a role-specialized multi-agent pipeline for schematic generation, contract construction, and testbench synthesis. Qualification requires the reference RTL/

testbench pair to elaborate, terminate, and satisfy the expected checks under Icarus.

- We conduct a matched 2×2 comparison of checkpoint adaptation and generated-schematic input. On VerilogEval-Human pass@1, the image-input difference is -3.01 points for the base checkpoint and +8.46 points for the adapted checkpoint; interactions at larger cutoffs and on RTLLM are smaller or mixed.

2 Related Work

We compare CIRCUITWEAVE with prior work on language-based RTL generation, multimodal generation, structured circuit representations, and executable verification.

2.1 Language-Based RTL Generation

Language-based RTL systems span English-to-Verilog translation, domain adaptation, and reasoning-enhanced generation. DAVE studied English-to-RTL translation [14]; VeriGen studied domain adaptation [18]; and CodeV constructed hierarchical instruction data from RTL repositories [25]. VeriThoughts combines reasoning data with formal-verification labels [22]; ReasoningV uses adaptive hybrid reasoning [15]; and VeriReason, VeriRL, and CodeV-R1 use simulator feedback or reinforcement learning [17, 19, 26]. These systems operate on language, code, reasoning, or EDA traces. CIRCUITWEAVE instead inserts a specification-side circuit contract before RTL generation.

2.2 Multimodal RTL Generation

Hardware specifications use circuit, timing, and FSM diagrams to encode relations that can be cumbersome to state in prose. Chang et al. report cases in which language omits information needed to recover the intended hardware [1]. VeriGround reports that blank-image and anonymized-identifier controls can reveal circuit-to-RTL improvements without corresponding visual grounding [20]. CIRCUITWEAVE instead maps each modality to a source-specific contract and uses matched checkpoint-by-input conditions. Because the present study lacks these grounding controls, its image deltas measure conditioning rather than verified visual grounding.

2.3 Structured Circuit Representations

Prior structure-aware methods constrain hardware generation without image input. CraftRTL uses correct-by-construction non-image

tasks [9], whereas CPPL lowers a typed circuit IR under legality checks [21]. CIRCUITWEAVE does not replace RTL with a compiler IR. Its specification-side contracts retain separate schematic-topology and textual-behavior records before fusion; the fused record then conditions RTL generation.

2.4 Executable Verification and Adaptation

Executable tests are a standard correctness signal in code generation [2], while RTL testbenches can check top-level interfaces and the cycle-level behavior they cover. VerilogEval and RTLMLM evaluate generated RTL by simulation [8, 11], and LLM4DV generates verification stimuli [24]. VeriReason and VeriRL use testbench or simulation feedback [17, 19]; QiMeng-SALV introduces signal-aware learning [23]; and CodeV-R1 combines reasoning supervision with reinforcement learning [26]. Before training, CIRCUITWEAVE executes each reference RTL/testbench pair; qualification means that the pair elaborates, terminates, and satisfies the expected checks. During benchmark evaluation, generated RTL is accepted only if it compiles and passes simulation against the hidden testbench.

3 CircuitWeave: Contract-Mediated Multimodal RTL Generation

CIRCUITWEAVE decomposes the pipeline into contract construction, contract fusion, and RTL generation with executable acceptance. Separate source-restricted calls predict a topology contract from the schematic and a behavior contract from the text. Their fused contract is the only serialized input supplied to the RTL-generation call. Training combines losses for the two source-specific contracts, serialized fusion, RTL generation, and reverse RTL-to-contract reconstruction. Figure 1 summarizes this evidence flow. The following subsections define the source-restricted calls, fused-only RTL decoder, training objective, and executable boundary.

3.1 Problem Formulation

Let x_t denote a textual specification and x_v a generated schematic. Let z_t and z_v denote the behavior and topology contracts, respectively, and let z be the fused circuit contract decoded from $p_\theta(z | z_v, z_t)$. Finally, let y denote the serialized RTL response containing the generated Verilog module. We parameterize the conditional model according to the following source-restricted inference graph:

$$\begin{aligned} p_\theta(z_v, z_t, z, y | x_v, x_t) \\ = p_\theta(z_v | x_v) p_\theta(z_t | x_t) p_\theta(z | z_v, z_t) p_\theta(y | z). \end{aligned} \quad (1)$$

Equation 1 encodes the designed conditional dependencies among evidence extraction, contract fusion, and code generation. Under this parameterization, marginalizing the intermediate contract sequences gives

$$\begin{aligned} p_\theta(y | x_t, x_v) = \sum_{z_v, z_t, z} p_\theta(y | z) p_\theta(z | z_v, z_t) \\ \cdot p_\theta(z_v | x_v) p_\theta(z_t | x_t). \end{aligned} \quad (2)$$

At inference, we do not enumerate all intermediate contracts in Equation 2. Instead, we follow the factorization through two phases and four separate decoder calls. In the contract phase, the topology call receives only x_v , the behavior call receives only x_t , and the fusion call receives only z_v, z_t . In the RTL phase, the fourth

Table 1: Artifact source and role in each package. Contracts supervise generation; only the reference RTL/testbench pair is EDA-executed.

Artifact	Construction source	Learning/validation role
Text	Web collection	Source for behavior contract z_t
Schematic	Schematic agent	Source for topology contract z_v
Contract record	Contract agent	Targets z_v, z_t , and fused z
RTL	Web collection	Target y ; compiled and simulated
Testbench	Verification agent	Hidden qualifier; executed with RTL

call receives only serialized z and emits y . Thus, the raw text and image are not supplied directly to the RTL-generation call. Source restriction denotes this call-level input constraint, not statistical independence between the modalities. When the image is absent, z_v contains explicit unknown fields instead of inferred visual evidence.

3.2 Contract-Annotated Executable Package Construction

Each of 5,000 design-verification packages contains the five artifact classes in Table 1. The contract record contains separate z_v, z_t , and z sections; the testbench remains a hidden qualification artifact and is never a model input.

We construct each design-verification package through a role-specialized multi-agent pipeline. Starting from a publicly accessible Verilog module and its natural-language description, a schematic agent translates the design intent into a visual representation. A contract agent then receives only its designated input in each pass: the topology pass receives the schematic, the behavior pass receives the description, and the fusion pass receives the two resulting contracts. During contract construction, this agent receives neither the reference RTL nor the testbench. These call-level restrictions remove the agent’s direct access to the reference artifacts. In parallel, a verification agent synthesizes a self-checking testbench from the textual specification. An executable validator finally compiles and simulates the reference RTL/testbench pair, retaining only packages that elaborate, terminate, and satisfy the expected checks.

Nano Banana 2, Gemini 3 Flash, and GPT-5.5 instantiate the schematic, contract, and verification roles, respectively [4, 5, 13]. The executable validator uses Icarus Verilog, invoking `iverilog` for compilation and elaboration and `vvp` for simulation [7].

This executable gate retains reference RTL/testbench pairs that elaborate and pass simulation; it does not establish test coverage, cross-modal fidelity, or semantic correctness beyond the exercised checks. The fusion and reverse-reconstruction losses are training signals rather than certification. We therefore call the retained packages *executable-qualified*. After normalizing module identifiers, contract headings, code fences, answer tags, and simulator outcomes, we screen within-corpus duplicates and split 4,750/250 packages for training/validation. The available records do not support a benchmark-aware semantic-overlap count, which we treat as a validity threat in Section 6.

Table 2 compares this package design with representative RTL-training resources. Each package in this work co-locates a generated schematic, modality-specific and fused contracts, reference RTL, and an executable testbench.

Table 2: Representative RTL-generation training resources. “TB use” describes testbenches used in curation or learning, not necessarily model inputs. Scale is rounded from the cited releases.

Resource	Scale	Learning target	Image	TB use	Pair-level quality control
RTLCoder-27K [10]	27K	Instruction + RTL SFT	No	No	Syntax checking
CodeV-Verilog [25]	165K	Multilevel summaries + RTL SFT	No	No	Syntax, deduplication, and decontamination
CraftRTL [9]	110K	Structured tasks + RTL/repair	No [†]	No	Construction checks and filtered repair
OriGen [3]	222K	RTL augmentation + repair	No	No	Compiler-guided syntax repair
VeriThoughts [22]	20K	Reasoning + RTL SFT	No	No	Formal-equivalence labels
CodeV-R1 [26]	87K+3.1K	Reasoning SFT + RL	No	Filter/RL	Generated-testbench equivalence
Veribench-53K [17]	53K	Reward modeling + RL	No	5–10/pair	Multi-testbench simulation
This work	5K	Topology/behavior/fused contracts + RTL	Yes	1/pair	Icarus compilation and simulation

[†] CraftRTL includes K-maps, FSMs, and waveforms as structured non-image representations. Executable QC does not imply full semantic certification.

Table 3: Contract fields and their treatment in fused z and reverse-reconstruction target z_{cov} .

Group	Topology z_o	Behavior z_t	Treatment in z / z_{cov}
Interface	Visible pins and links	Name, direction, width	z : fuse ports; z_{cov} : declarations
Sequential	Registers, state edges	Clock, reset, latency	z : fuse state/timing; z_{cov} : event controls
Datapath	Operators, dependencies	Function, corner cases	z : fuse operations; z_{cov} : selected dependencies
Status	Visible or unknown	Stated or unknown	z : preserve source/status; z_{cov} : omit unknown/conflict

3.3 Topology–Behavior Contract Fusion

The topology contract z_o assigns the schematic a structural role. Its fixed headings cover components and ports, connectivity, datapath operators, register boundaries, state transitions, internal dependencies, and unknown evidence. The behavior contract z_t assigns the text a complementary role and records the module interface, widths, clock/reset semantics, function, latency, output timing, and corner cases. Here, source isolation is a call-level restriction: the topology pass receives only x_o , and the behavior pass receives only x_t . It removes direct access to the other raw input before fusion but does not imply provenance or statistical independence.

The fusion target aligns shared entities, carries forward modality-specific constraints, and labels shared fields as supported, missing, or conflicting. Explicit textual interface and timing requirements fill fields absent from the schematic, while visible structural relations retain their z_o provenance. The resulting z serializes the cross-modal correspondence and the constraints supplied to the RTL-generation call. Listing 1 shows the two-phase serialization used by the model.

Table 3 summarizes the contract groups and their fusion policy. The serialized schema stores each field value with its evidence source. In the target records, unsupported fields are labeled unknown. When two explicit values disagree, the field remains conflict unless a stated field-specific authority rule applies; any unresolved conflict remains explicit in z . These labels distinguish missing from conflicting evidence instead of leaving both implicit in a free-form rationale.

Listing 1: Source-restricted contract calls and fused-only RTL call.

```

Contract phase:
A. schematic only -> <topology> components; edges; operators; state </topology>
B. text only -> <behavior> ports/widths; clock/reset; timing; function </behavior>
C. z_v and z_t only -> <fused> correspondence; missing/conflict; RTL constraints </fused>
RTL phase:
D. fused z only -> <answer> complete Verilog module </answer>

```

3.4 Contract-Conditioned RTL Generation

We train the shared decoder on five conditional tasks with length-normalized token losses. Let $z_{\text{cov}} = P_{\text{cov}}(z)$ be a deterministic projection of z . It retains port declarations, register/state elements, clock/reset event controls, and explicit datapath operators and dependencies. The projection omits unknown/conflict values and behavioral requirements outside the scope of the reverse task, including latency, corner-case behavior, and full functional equivalence. The losses $\mathcal{L}_{\text{topo}}$, $\mathcal{L}_{\text{behavior}}$, and $\mathcal{L}_{\text{RTL}}$ predict z_o from x_o , z_t from x_t , and reference RTL y from fused z , respectively. The serialized fusion loss $\mathcal{L}_{\text{align}} = -\log p_{\theta}(z \mid z_o, z_t)$ supervises the target fused record, including its correspondence and status fields. The reverse-reconstruction loss $\mathcal{L}_{\text{cons}} = -\log p_{\theta}(z_{\text{cov}} \mid y)$ predicts z_{cov} from reference RTL and supplies an indirect auxiliary signal through the shared parameters. The complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{topo}} + \mathcal{L}_{\text{behavior}} + \mathcal{L}_{\text{RTL}} + \lambda \mathcal{L}_{\text{align}} + \mu \mathcal{L}_{\text{cons}}. \quad (3)$$

Because z explicitly serializes field correspondences and their status, $\mathcal{L}_{\text{align}}$ supervises those targets but is not an independent measurement of alignment accuracy. We use LoRA for parameter-efficient adaptation of the base multimodal model. The training scheme combines contract-mediated generation with a reverse-reconstruction auxiliary objective. Concretely, trainable low-rank updates are inserted into the Qwen3.5-4B language-model linear layers while the base weights, vision encoder, and multimodal aligner remain frozen [6, 12]. No benchmark testbench or simulator outcome is exposed during training.

Each term has a teacher-forced gradient path to the shared decoder parameters. The contract-phase terms supervise the two source-specific serializations and their fusion; the RTL term supervises $z \rightarrow y$; and the reverse term supervises $y \rightarrow z_{\text{cov}}$. Training

supplies the reference fused contract to the RTL task, whereas inference supplies the predicted contract from the contract phase. Because the reverse task does not compare a sampled $\hat{y}$ with z , it regularizes the shared parameters only indirectly and does not enforce per-candidate contract-RTL consistency. It is neither an EDA reward nor a claim of formal equivalence.

3.5 Candidate Extraction and Executable Acceptance

All base and adapted variants receive the same two-stage and output-format instructions. Let $E(y)$ extract a complete module from the final `<answer>` block and return $\perp$ otherwise. The evaluator removes response markup but does not repair code. It compiles $E(y)$ with hidden testbench T and simulates each successfully elaborated candidate:

$$A(y, T) = \mathbb{1}[E(y) \neq \perp \wedge \text{compile}(E(y), T) \wedge \text{simulate}(E(y), T)]. \quad (4)$$

Extraction failures, interface mismatches, compilation errors, timeouts, and testbench-detected functional mismatches all receive zero. The reverse auxiliary task reconstructs selected contract fields from reference RTL during training, whereas the hidden testbench defines the final benchmark acceptance decision for generated candidates. A passing result establishes only satisfaction of the behavior exercised by T . Every condition uses the same extraction and EDA procedure without checker-based reranking or post-generation repair.

4 Experimental Setup

The experiments cross checkpoint state with inference condition in a matched 2×2 design. **RQ1** quantifies the end-to-end difference between the base checkpoint and the checkpoint adapted with the complete contract-mediated objective. **RQ2** quantifies the within-checkpoint difference associated with adding a description-derived schematic. **RQ3** examines whether the resulting descriptive interaction has a consistent direction across benchmarks and sampling cutoffs.

4.1 Benchmarks, Conditions, and Metric

We evaluate VerilogEval-Human (156 tasks) [8] and RTLML [11], both of which provide textual tasks and functional testbenches. For each task and condition, we draw 20 samples at temperature 0.7 and compile and simulate every candidate independently. We report $\text{pass}@1$, $\text{pass}@5$, and $\text{pass}@10$ using the standard unbiased estimator of Chen et al. [2], averaged across tasks without category reweighting. A candidate must satisfy Eq. 4; similarity to reference RTL is not used.

The four conditions in Table 4 cross checkpoint state (base or adapted) with contract input (text or text+image). In the text+image condition, Nano Banana 2 generates a schematic from the released task description alone and does not receive the reference RTL or benchmark testbench. In text-only inference, the topology contract uses explicit unknown values; in text+image inference, it is extracted from the generated schematic. The same LoRA adapter is evaluated

in both modes. Every row follows the same contract-first, RTL-second protocol, sampling budget, extraction procedure, and EDA harness, without reranking or repair.

4.2 Training and Comparison Protocol

We train on 4,750 packages and validate on 250. Training optimizes Eq. 3, with each term normalized by its supervised token count. Separate source-restricted examples implement the two contract branches, and the reference fused contract conditions $\mathcal{L}_{\text{RTL}}$ during teacher forcing. The base checkpoint is Qwen3.5-4B [16]. Training uses bfloat16 on one GPU and the ms-swift all-linear LoRA target [12]. Rank is 32 with scaling factor 64. We train for three epochs at learning rate 1×10^{-4} , warmup ratio 0.05, and sequence length 8192. Per-device batch size is 2 with gradient accumulation 2, and the image-token budget is 1024. Evaluation and checkpointing occur every 100 steps, with two checkpoints retained.

External systems in Table 4 differ in prompting, model scale, decoding, benchmark revision, and post-processing. Their reported values provide approximate context for the compact 4B model rather than head-to-head comparisons. All claims about adaptation and image input rely on the four controlled rows.

5 Results and Analysis

5.1 RQ1: Base-to-Adapted Performance

Across both benchmarks and all three k values, the adapted checkpoint has higher point estimates than the corresponding base checkpoint. At $k = 1, 5, 10$, text-only VerilogEval-Human $\text{pass}@k$ rises by 14.55, 14.23, and 14.74 points to 38.14%, 55.64%, and 62.82%. With text+image, the corresponding differences are 26.02, 18.52, and 14.11 points, reaching 46.60%, 61.49%, and 65.39%. On RTLML, the base-to-adapted differences are 10, 11, and 9 points for text and 13, 9, and 8 points for text+image. These comparisons evaluate the combined adaptation recipe, including all five objective terms and LoRA; they do not identify the contribution of any individual branch, loss, or adaptation component.

5.2 RQ2-RQ3: Checkpoint-Input Interaction

The within-checkpoint deltas in Table 5 summarize the difference associated with adding the generated schematic. On VerilogEval-Human $\text{pass}@1$, the image delta changes from -3.01 points in the base model to $+8.46$ after adaptation, giving a descriptive interaction of 11.47 points. The corresponding interaction is $+4.29$ at $\text{pass}@5$ but -0.63 at $\text{pass}@10$. On RTLML, the adapted image delta is consistently $+2$ points, while the interactions are $+3$, -2 , and -1 points at $\text{pass}@1$, $\text{pass}@5$, and $\text{pass}@10$. The interaction is largest at the lower VerilogEval-Human cutoffs and does not have a consistent direction across benchmarks or sampling cutoffs.

The reported external rows contextualize scale but do not form a unified leaderboard. In the reported values, CIRCUITWEAVE is above StarCoder2-15B at every VerilogEval-Human cutoff and on all three RTLML metrics; because the protocols differ, these are contextual rather than head-to-head comparisons. Several external systems also report higher values on parts of the table. Without blank-image, mismatched-image, or identifier-anonymization controls, the positive image-minus-text deltas for the adapted checkpoint are consistent with generated-schematic conditioning but do not verify

Table 4: Executable RTL-generation results (%). Controlled rows share decoding and EDA settings; bold marks the best controlled value for each metric. Reported external rows provide scale context only because protocols differ.

Group	Model	Input	VerilogEval-Human			RTLML		
			pass@1	pass@5	pass@10	pass@1	pass@5	pass@10
Controlled	Base Qwen3.5-4B	Text	23.59	41.41	48.08	28.00	35.00	41.00
Controlled	Base Qwen3.5-4B	Text+image	20.58	42.97	51.28	27.00	39.00	44.00
Controlled	CIRCUITWEAVE w/o image	Text	38.14	55.64	62.82	38.00	46.00	50.00
Controlled	CircuitWeave	Text+image	46.60	61.49	65.39	40.00	48.00	52.00
Reported	GPT-4o [9]	Text	57.1	63.9	66.7	33.8	44.4	48.3
Reported	DeepSeek-V3 [23]	Text	70.7	77.4	78.8	62.0	72.0	72.4
Reported	StarCoder2-15B [9]	Text	37.7	50.6	57.2	15.5	37.6	45.7
Reported	CodeV-CodeQwen [9]	Text	53.2	65.1	68.5	36.6	53.3	61.3
Reported	OriGen [23]	Text	54.4	60.1	64.2	50.6	68.3	74.3
Reported	CodeV-R1-7B [26]	Text	69.9	79.3	81.7	72.9	86.1	—

Table 5: Image-input deltas (generated schematic minus text only, percentage points). Interaction is the adapted delta minus the base delta; all values are descriptive. “VerilogEval” denotes VerilogEval-Human.

Benchmark	k	Base Δ	Adapted Δ	Interaction
VerilogEval	1	-3.01	+8.46	+11.47
	5	+1.56	+5.85	+4.29
	10	+3.20	+2.57	-0.63
RTLML	1	-1.00	+2.00	+3.00
	5	+4.00	+2.00	-2.00
	10	+3.00	+2.00	-1.00

visual grounding [20]. Task-level bootstrap intervals and stage-wise failure counts are unavailable, so all differences remain descriptive, with uncertainty especially consequential for the smaller deltas. Under the shared EDA protocol, the adapted checkpoint has positive image-minus-text differences on all reported metrics, whereas evidence for a stable adaptation-by-image interaction is mixed.

5.3 Qualitative FSM Case Study

Figure 2 compares outputs from the same adapted Qwen3.5-4B checkpoint on the same FSM prompt. The model and text are fixed; one condition also receives the generated schematic. Both candidates recover the reset and transitions. The schematic specifies $Z = Q$, which the image-conditioned candidate implements as `assign out = state`. The text-only candidate updates out in the clocked block; nonblocking semantics therefore use the pre-edge state and cause a one-cycle lag on both transitions. This paired example illustrates how schematic conditioning can help the same model avoid a concrete RTL error and complements the adapted checkpoint’s positive aggregate image-minus-text deltas in Tables 4 and 5.

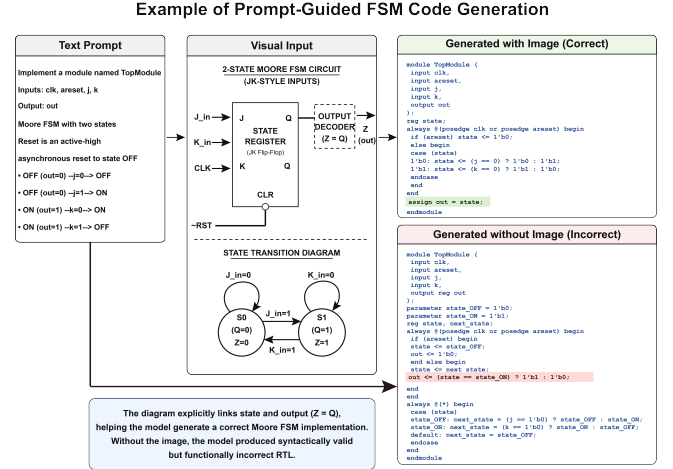


Figure 2: Paired outputs from the same adapted Qwen3.5-4B checkpoint on one FSM task. The image-conditioned candidate uses the current state; the text-only candidate uses the pre-transition state and introduces a one-cycle lag. Highlights mark the assignments.

6 Limitations

The present study is limited to description-derived schematics, one 4B checkpoint, and simulation-level evaluation. Passing Icarus qualification establishes agreement between the reference RTL and testbench only for the exercised behaviors; it does not establish coverage or cross-modal fidelity, and weak tests may accept incorrect RTL. Web collection also creates provenance, licensing, and benchmark-overlap risks. The reported results are aggregate point estimates without repeated runs across training or sampling seeds, paired task-level intervals, or stage-wise failure counts. Evaluation excludes synthesis, timing, area, power, and physical design.

The controlled design compares checkpoint state and the addition of a description-derived schematic; it does not isolate the individual contributions of the contract branches, $\mathcal{L}_{\text{align}}$, $\mathcal{L}_{\text{cons}}$, or LoRA. Generated contracts have not been independently audited,

and the schematics derive from the same text used by the text-only condition. Without blank, mismatched, anonymized, or topology-perturbed image controls, the results quantify conditioning on these generated schematics rather than verified visual grounding. Future evaluation should include objective ablations, independently authored diagrams, broader contract coverage, and coverage-guided tests.

7 Conclusion

We presented CIRCUITWEAVE, which fuses schematic topology and textual behavior into a contract before RTL decoding. LoRA adaptation of Qwen3.5-4B uses the training split of a 5,000-package executable-qualified corpus. Under the text+image condition, the adapted model reaches 46.60% pass@1 on VerilogEval-Human and 40.00% on RTLLM. On VerilogEval-Human pass@1, the image-input difference is -3.01 points for the base checkpoint and $+8.46$ points for the adapted checkpoint. Under the shared evaluation protocol, these point estimates characterize the end-to-end performance of contract-mediated multimodal RTL generation; they do not isolate individual adaptation components or establish visual grounding.

References

- [1] Kaiyan Chang, Zhirong Chen, Yunhao Zhou, Wenlong Zhu, Kun Wang, Haobo Xu, Cangyuan Li, Mengdi Wang, Shengwen Liang, Huawei Li, Yinhe Han, and Ying Wang. 2024. Natural Language is Not Enough: Benchmarking Multi-modal Generative AI for Verilog Generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*. Association for Computing Machinery, New York, NY, USA, 117:1–117:9. doi:10.1145/3676536.3676679
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374. doi:10.48550/arXiv.2107.03374
- [3] Fan Cui, Chenyang Yin, Kexing Zhou, Youwei Xiao, Guangyu Sun, Qiang Xu, Qipeng Guo, Yun Liang, Xingcheng Zhang, Demin Song, and Dahua Lin. 2024. OriGen: Enhancing RTL Code Generation with Code-to-Code Augmentation and Self-Reflection. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. Association for Computing Machinery, New York, NY, USA, 99:1–99:9. doi:10.1145/3676536.3676830
- [4] Google. 2026. Gemini 3 Flash Preview. <https://ai.google.dev/gemini-api/docs/models/gemini-3-flash-preview>. Endpoint: gemini-3-flash-preview; accessed: 2026-07-18.
- [5] Google DeepMind. 2026. Nano Banana 2: Combining Pro Capabilities with Lightning-Fast Speed. <https://blog.google/innovation-and-ai/technology/ai/nano-banana-2/>. Endpoint: gemini-3.1-flash-image-preview; accessed: 2026-07-18.
- [6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. OpenReview.net, Virtual Event, 26 pages. <https://openreview.net/forum?id=nZevKeeFYf9>
- [7] Icarus Verilog Project. 2026. Icarus Verilog Documentation. <https://steveicarus.github.io/iverilog/>. Accessed: 2026-07-10.
- [8] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *Proceedings of the 2023 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, San Francisco, CA, USA, 1–8. doi:10.1109/ICCAD57390.2023.10323812
- [9] Mingjie Liu, Yun-Da Tsai, Wenfei Zhou, and Haoxing Ren. 2025. CraftRTL: High-quality Synthetic Data Generation for Verilog Code Models with Correct-by-Construction Non-Textual Representations and Targeted Code Repair. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. OpenReview.net, Singapore, 90377–90422. https://proceedings.iclr.cc/paper_files/paper/2025/hash/e112a4671e8779aa9f640a0e3f81bd26-Abstract-Conference.html
- [10] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2025. RTLcoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 4 (2025), 1448–1461. doi:10.1109/TCAD.2024.3483089
- [11] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *Proceedings of the 29th Asia and South Pacific Design Automation Conference*. IEEE, Incheon, Republic of Korea, 722–727. doi:10.1109/ASP-DAC58780.2024.10473904
- [12] ModelScope Team. 2026. MS-Swift: Command-Line Parameters. Software documentation. Accessed: 2026-07-11. https://github.com/modelscope/ms-swift/blob/main/docs/source_en/Instruction/Command-line-parameters.md
- [13] OpenAI. 2026. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>. Accessed: 2026-07-10.
- [14] Hammond Pearce, Benjamin Tan, and Ramesh Karri. 2020. DAVE: Deriving Automatically Verilog from English. In *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*. Association for Computing Machinery, New York, NY, USA, 27–32. doi:10.1145/3380446.3430634
- [15] Haiyan Qin, Zhiwei Xie, Jingjing Li, Liangchen Li, Xiaotong Feng, Junzhan Liu, and Wang Kang. 2025. ReasoningV: Efficient Verilog Code Generation with Adaptive Hybrid Reasoning Model. arXiv:2504.14560. doi:10.48550/arXiv.2504.14560
- [16] Qwen Team. 2026. Qwen3.5-4B Model Card. Hugging Face model repository. Accessed: 2026-07-11. <https://huggingface.co/Qwen/Qwen3.5-4B>
- [17] Fu Teng, Miao Pan, Xuhong Zhang, Zhezhi He, Yiyao Yang, Xinyi Chai, Mengnan Qi, Liqiang Lu, and Jianwei Yin. 2025. VeriRL: Boosting the LLM-based Verilog Code Generation via Reinforcement Learning. In *2025 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, Piscataway, NJ, USA, 1–9. doi:10.1109/ICCAD66269.2025.11241003
- [18] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 46:1–46:31. doi:10.1145/3643681
- [19] Yiting Wang, Guoheng Sun, Wanghao Ye, Gang Qu, and Ang Li. 2025. VeriReason: Reinforcement Learning with Testbench Feedback for Reasoning-Enhanced Verilog Generation. arXiv:2505.11849.
- [20] Guang Yang, Xing Hu, Xiang Chen, and Xin Xia. 2026. From Mirage to Grounding: Towards Reliable Multimodal Circuit-to-Verilog Code Generation. arXiv:2604.27969. doi:10.48550/arXiv.2604.27969
- [21] Shuo Yin, Yihe Wang, Lancheng Zou, Xufeng Yao, Tinghuan Chen, Chen Bai, Zhengrong Wang, Tsung-Yi Ho, and Bei Yu. 2026. CPPL: A Circuit Prompt Programming Language. arXiv:2605.17892.
- [22] Patrick Yubeaton, Andre Nakkab, Weihua Xiao, Luca Collini, Ramesh Karri, Chinmay Hegde, and Siddharth Garg. 2025. VeriThoughts: Enabling Automated Verilog Code Generation Using Reasoning and Formal Verification. In *Advances in Neural Information Processing Systems 38: Datasets and Benchmarks Track*. Curran Associates, Inc., Red Hook, NY, USA, 23 pages.
- [23] Yang Zhang, Rui Zhang, Jiaming Guo, Lei Huang, Yunpu Zhao, Shuyao Cheng, Pengwei Jin, Chongxiao Li, Zidong Du, Xing Hu, Qi Guo, and Yunji Chen. 2025. QiMeng-SALV: Signal-Aware Learning for Verilog Code Generation. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., Red Hook, NY, USA, 159613–159633. https://papers.neurips.cc/paper_files/paper/2025/hash/ea052d3e98adfeaf0fbff1766ee2efde-Abstract-Conference.html
- [24] Zixi Zhang, Balint Szekely, Pedro Gimenes, Greg Chadwick, Hugo McNally, Jianyi Cheng, Robert Mullins, and Yiren Zhao. 2025. LLM4DV: Using Large Language Models for Hardware Test Stimuli Generation. In *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Piscataway, NJ, USA, 133–137. doi:10.1109/FCCM62733.2025.00048
- [25] Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yanan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, Yansong Pan, Zhenxing Zhang, Rui Zhang, Xishan Zhang, Zidong Du, Qi Guo, and Xing Hu. 2026. CodeV: Empowering LLMs with HDL Generation through Multilevel Summarization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 4 (2026), 1893–1906. doi:10.1109/TCAD.2025.3604320
- [26] Yaoyu Zhu, Di Huang, Hanqi Lyu, Xiaoyun Zhang, Chongxiao Li, Wenxuan Shi, Yutong Wu, Jianan Mu, Jinghua Wang, Yang Zhao, Pengwei Jin, Shuyao Cheng, Shengwen Liang, Xishan Zhang, Rui Zhang, Zidong Du, Qi Guo, Xing Hu, and Yunji Chen. 2025. QiMeng-CodeV-R1: Reasoning-Enhanced Verilog Generation. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., Red Hook, NY, USA, 154266–154300. https://papers.nips.cc/paper_files/paper/2025/hash/e2f7407b62f152b7fe533fbc077fddb7-Abstract-Conference.html